🚀 PublicSpark — Full-Stack Deployment, Code Separation & Multi-DB Architecture Guide

Project: PublicSpark Digital Newsroom Platform **Target Environment:** Production Hosting (Vercel/Netlify for Frontend, Render/Railway/VPS for Backend) **Database Engines:** Supabase PostgreSQL + Turso Edge SQLite + Cloudflare R2 **Document Date:** October 10, 2026

📑 Table of Contents

1. [Executive Summary](#1-executive-summary) 2. [Frontend vs. Backend Code Separation](#2-frontend-vs-backend-code-separation) 3. [Frontend Deployment Procedure](#3-frontend-deployment-procedure) 4. [Backend Deployment Procedure](#4-backend-deployment-procedure) 5. [Connecting Frontend to Production Backend URL](#5-connecting-frontend-to-production-backend-url) 6. [Multi-DB Connection & Architecture Procedure](#6-multi-db-connection--architecture-procedure) 7. [Environment Variables (`.env`) Master Template](#7-environment-variables-env-master-template) 8. [End-to-End Code Examples](#8-end-to-end-code-examples)

---

1. Executive Summary

PublicSpark is architected as a **decoupled full-stack web platform**.

- **Frontend (Client-Side)**: Handles user interaction, responsive newsroom UI, client caching, theme switching, 1-click share modals, and bookmarks.
- **Backend (Server-Side)**: Handles Text-to-Speech synthesis (Google Cloud TTS & Sarvam AI), 18-language multi-tier translation API proxy, analytics view tracking, and server database persistence.
- **Multi-DB Layer**: Employs a zero-cost hybrid database strategy splitting Master relational data (Supabase PostgreSQL), fast edge caches (Turso SQLite), and media storage (Cloudflare R2).

---

2. Frontend vs. Backend Code Separation

🌐 Frontend (Client-Side App)

These files execute strictly inside the user's browser.

| File / Folder Path | Purpose | | :--- | :--- | | `src/App.jsx` | Main application shell, state router, view switcher, footer social icons. | | `src/components/common/` | Header with 4s ticker, loading splash, toast notices. | | `src/components/public/` | ArticleReader, SearchPage, Share modal popup. | | `src/components/admin/` | AdminWorkspace, Analytics dashboard, Storage settings, Social links desk. | | `src/components/reporter/` | ReporterWorkspace, WebP image compressor form. | | `src/components/editor/` | EditorWorkspace, story approval & rejection desk. | | `src/services/` | `cacheService.js`, `translationService.js`, `speechEngine.js`, `mediaService.js`. | | `public/` | Static branding assets (`logo.png`). | | `index.html` | Base HTML document, Google Fonts preloader. | | `vite.config.js` | Vite build tool configuration. |

⚙️ Backend (Server-Side App)

These files execute strictly on your Node.js cloud server.

| File / Folder Path | Purpose | | :--- | :--- | | `server/server.js` | Native Node.js HTTP Proxy Server, API routes, TTS, analytics, CORS, storage endpoints. | | `server/data/settings.json` | Server-persisted social channels configuration file. | | `server/data/categories.json` | Server-persisted news category taxonomy file. | | `.env` | Secret API keys (Google Cloud TTS, Sarvam AI, Supabase, Turso, Cloudflare R2). |

---

3. Frontend Deployment Procedure

Option A: Vercel (Recommended)

1. Push your repository to GitHub. 2. Sign in to [Vercel](https://vercel.com) and click **"New Project"**. 3. Import your GitHub repository `pintu-chachu-news-web`. 4. Vercel automatically detects Vite: - **Framework Preset**: Vite - **Build Command**: `npm run build` - **Output Directory**: `dist` 5. Click **Deploy**. Vercel will give you a domain like `https://publicspark.vercel.app`.

Option B: Netlify

1. Log in to [Netlify](https://netlify.com) and select **"Add new site"** -> **"Import an existing project"**. 2. Select GitHub and pick the repository. 3. Configure settings: - **Build command**: `npm run build` - **Publish directory**: `dist` 4. Click **Deploy site**.

---

4. Backend Deployment Procedure

Option A: Render.com (Recommended Free Tier)

1. Sign in to [Render](https://render.com) and select **"New Web Service"**. 2. Connect your GitHub repository. 3. Configure settings: - **Environment**: Node - **Build Command**: `npm install` - **Start Command**: `node server/server.js` 4. In **Environment Variables**, add: - `PORT` = `5000` - `GOOGLE_CLOUD_TTS_API_KEY` = *your_key* - `SARVAM_AI_API_KEY` = *your_key* - `SUPABASE_URL` = *your_url* - `SUPABASE_SERVICE_ROLE_KEY` = *your_key* 5. Click **Create Web Service**. Render will give you a backend URL like `https://publicspark-api.onrender.com`.

---

5. Connecting Frontend to Production Backend URL

In development, the frontend calls `http://localhost:5000`. In production, update the API base URL in your frontend config:

Create or update `src/config.js`: ``` `javascript export const API_BASE_URL = process.env.NODE_ENV === 'production' ? 'https://publicspark-api.onrender.com' : 'http://localhost:5000'; `

`Then in src/App.jsx and services: `javascript import { API_BASE_URL } from './config';

`// Example Backend Call: fetch(${API_BASE_URL}/api/settings/social) .then(res => res.json()) .then(data => setSocialLinks(data)); `

---

6. Multi-DB Connection & Architecture Procedure

Our hybrid database strategy splits tasks across 3 dedicated services to achieve **₹0 direct cost**:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        PUBLICSPARK HYBRID DB LAYER                       │
├──────────────────────────┬──────────────────────────┬────────────────────┤
│        Supabase          │     Turso (Edge SQLite)  │    Cloudflare R2   │
│   (PostgreSQL)           │                          │                    │
├──────────────────────────┼──────────────────────────┼────────────────────┤
│ Master Relational Data   │ 18-Language Fast Cache   │ Media Storage      │
│ Articles, Categories,    │ Translation hashes &     │ Generated MP3 &    │
│ Profiles, RLS Auth       │ text cache               │ WebP Images        │
└──────────────────────────┴──────────────────────────┴────────────────────┘
```

Step 1: Install Database SDK Dependencies

```bash
npm install @supabase/supabase-js @libsql/client @aws-sdk/client-s3
```

Step 2: Create Multi-DB Connector (server/db.js)

```javascript
import { createClient as createSupabaseClient } from '@supabase/supabase-js';
import { createClient as createTursoClient } from '@libsql/client';
import { S3Client } from '@aws-sdk/client-s3';

// 1. Supabase Master PostgreSQL Client
export const supabase = createSupabaseClient( process.env.SUPABASE_URL, process.env.SUPABASE_SERVICE_ROLE_KEY );

// 2. Turso Edge SQLite Translation Cache Client
export const turso = createTursoClient({ url: process.env.TURSO_DATABASE_URL, authToken: process.env.TURSO_AUTH_TOKEN });

// 3. Cloudflare R2 Storage S3-Compatible Client
export const r2 = new S3Client({ region: 'auto', endpoint: https://${process.env.R2_ACCOUNT_ID}.r2.cloudflarestorage.com , credentials: { accessKeyId: process.env.R2_ACCESS_KEY_ID, secretAccessKey: process.env.R2_SECRET_ACCESS_KEY } });
```

Step 3: DB Usage Patterns in Backend Server

```javascript
// 1. Supabase Query Example (Master DB)
const { data: articles, error } = await supabase .from('articles') .select('*') .eq('status', 'published');

// 2. Turso Query Example (Translation Edge Cache)
const result = await turso.execute({ sql: 'SELECT translated_text FROM translation_cache WHERE text_hash = ? AND lang = ?', args: [textHash, langCode] });

// 3. Cloudflare R2 Upload Example (MP3 Audio)
await r2.send(new PutObjectCommand({ Bucket: process.env.R2_BUCKET_NAME, Key: audio/article_${articleId}_${langCode}.mp3 , Body: audioBuffer, ContentType: 'audio/mpeg' }));
```

---

7. Environment Variables (.env) Master Template

Save this template in .env at the root of your project:

` env

Node Backend Port

PORT=5000

Text-to-Speech & Translation API Keys

GOOGLE_CLOUD_TTS_API_KEY=AIzaSy... SARVAM_AI_API_KEY=sarvam_...

1. Supabase Master Database

SUPABASE_URL=https://your-project.supabase.co SUPABASE_SERVICE_ROLE_KEY=eyJhbGci...

2. Turso Edge SQLite Database

TURSO_DATABASE_URL=libsql://your-db.turso.io TURSO_AUTH_TOKEN=ey...

3. Cloudflare R2 Media Storage

R2_ACCOUNT_ID=your_account_id R2_ACCESS_KEY_ID=your_access_key
R2_SECRET_ACCESS_KEY=your_secret_key R2_BUCKET_NAME=publicspark-media `

8. Summary Checklist Before Launch

- [x] Separate Frontend React code (src/) from Backend server code (server/).
- [x] Configure backend endpoints for settings.json and categories.json server persistence.
- [x] Test production build using npm run build`.
- [x] Set up environment variables in production cloud host (Render / Vercel).
- [x] Update API base URL in frontend to point to live backend domain.